Nguyen Dinh Quang Khanh

Ho Chi Minh City, Vietnam | khanhndq2002@gmail.com | +84 377 492 045 | linkedin.com/in/ndqkhanh | github.com/ndqkhanh

SUMMARY

Backend Software Engineer with 2+ years building scalable distributed systems at 2–3K RPS in production. Delivered real-time event-driven platforms, reduced cloud costs by 90%, and architected fault-tolerant microservices. Deep expertise in Java concurrency, MVCC transaction engines, hybrid search (BM25 + HNSW vector), and agentic AI runtimes. CS graduate with 3.92/4.0 GPA (Salutatorian).

TECHNICAL SKILLS

Languages	Java, Python, TypeScript, SQL **Frameworks:** Vert.x, Spring Boot, React, React Native
Java Concurrency	ReentrantLock, ReadWriteLock, CAS/AtomicReference, ConcurrentHashMap, StampedLock, ForkJoinPool, CompletableFuture, Virtual Threads, StructuredTaskScope
Data & Streaming	Apache Kafka, Apache Flink, SparkSQL, HBase, StarRocks, Redis, dbt, PostgreSQL, RocksDB
Infrastructure	Docker, Kubernetes (AKS), Helm, Jenkins, Airflow, Terraform, CI/CD **Protocols:** gRPC, REST, GraphQL, SSE, MCP, JSON-RPC, A2A
Observability	Grafana, OpenTelemetry, Loki, Prometheus, Micrometer
Architecture	Microservices, EDA, DDD, CQRS, Saga, Outbox, MVCC, Raft Consensus, Non-blocking Algorithms, Lock-Free Data Structures, HNSW, BM25, RRF Hybrid Ranking, Agentic AI Runtimes

PROFESSIONAL EXPERIENCE

MoMo — Backend Software Engineer *Ho Chi Minh City | Jan 2025 – Present*

- Designed the **Journey Platform**, a real-time IFTTT engine processing **2–3K events/sec** to automate personalized user journeys—reward distribution, push notifications, and in-app messaging—for **30M+ monthly active users**; selected as one of **MoMo's Top 16 Projects of the Year**.
- Architected a **JSON-based rule engine** with AND/OR Boolean condition trees and **weighted short-circuit evaluation**: conditions are sorted by data-source cost (in-memory < Redis < external API), reducing rule evaluation latency by **90%+** by skipping expensive external calls when cheap conditions fail first.
- Built a **three-tier event deduplication** pipeline (Bloom filter → Redis 24h cache → MySQL persistent store) achieving **exactly-once processing** semantics on top of Kafka's at-least-once delivery, eliminating duplicate reward grants across 500M+ monthly transactions.
- Engineered an **event prefiltering** system using in-memory hash maps to route each event type to only its relevant campaigns (O(1) lookup), reducing CPU utilization by **30%** by skipping rule evaluation for unmatched campaign–event pairs across 500+ concurrent campaigns.
- Built **per-downstream-service rate limiting** with token bucket algorithm and time-of-day varying rates, preventing cascade failures to reward, messaging, and push notification services during peak traffic; implemented exponential backoff with jitter for retry resilience.
- Reduced BigQuery costs from **$100/day to under $10/day (∼90% savings)** by migrating high-volume campaign tracking queries to on-premise HBase with **batched increment writes** (accumulate in-memory via AtomicLong, flush periodically) reducing DB round-trips from 2K/sec to ∼1/sec.
- Integrated **Grafana + OpenTelemetry** dashboards tracking P50/P99 rule evaluation latency, dedup hit ratio, action success rate, and per-campaign budget burn; reduced MTTD by **60%** and enabled marketing teams to autonomously launch campaigns in hours instead of days.

PROJECTS

VectorForge: Distributed AI-Augmented Search Engine [github.com/ndqkhanh/vector-forge] *Apr 2025*

Java 21, HNSW, BM25, RRF Hybrid Ranking, Consistent Hashing, SWIM Gossip, Virtual Threads | 8 modules, 781 tests

- Built a **distributed hybrid search engine** (88 source files, **781 tests, 0 failures**) with a hand-rolled **HNSW vector index** (Malkov 2018) achieving 95%+ recall@10 and a **lock-free inverted index** with epoch-based GC, skip-list posting lists, and delta + variable-byte encoding for **60% compression**.
- Implemented **Reciprocal Rank Fusion** combining BM25 lexical and cosine vector scores with **MMR diversity reranking**; distributed **scatter-gather** query engine using Java 21 `StructuredTaskScope` for parallel shard queries with **two-tier caching** (exact + semantic similarity).
- **Product quantization** reducing vector memory **12x**; near-real-time **segment architecture** with tiered merge; **SWIM gossip** with shard-aware dissemination; LLM-powered query expansion via AgentForge. JMH: **50K+ docs/sec** indexing, **<5ms** P50 kNN latency at 100K vectors.

NexusDB: Adaptive Concurrency Transactional Engine [github.com/ndqkhanh/nexus-db] *Oct 2024*

Java 21 | CAS, ReentrantReadWriteLock, AtomicReference, StampedLock, Virtual Threads | WAL, B-Tree, MVCC

- Built a **transactional storage engine in Java** with a novel **adaptive lock elision** strategy: the engine continuously measures contention ratio per key range and **auto-switches** between optimistic CAS-based paths and pessimistic ReentrantLock 2PL—reducing lock overhead by up to 3x under skewed workloads vs. static locking.
- Implemented **MVCC** with AtomicReference version chains and **epoch-based garbage collection** (inspired by Linux RCU) that reclaims old versions without stop-the-world pauses, using StampedLock optimistic reads for safe concurrent traversal.
- Concurrent **B-Tree with hand-over-hand latch coupling** using ReadWriteLock for lock-free readers. **Write-Ahead Log** with ARIES-style redo/undo and group commit batching via Virtual Threads for I/O throughput.
- Full **SSI** (Serializable Snapshot Isolation) with stale-read detection + write-dependency tracking. Anomaly test suite triggering **dirty reads, write skew, phantom reads** at each isolation level. JMH: **80K+ txn/sec** under 64-thread contention; adaptive elision outperforms static 2PL by **3.2x** on hot keys. **133 tests, 0 failures**.

AgentForge: Agentic AI Runtime Engine [github.com/ndqkhanh/agent-forge] *Feb 2025*

Java 21, Sealed Interfaces, Records, Pattern Matching, Virtual Threads, SSE, MCP, JSON-RPC 2.0 | 8 modules, 736 tests

- Engineered an **8-module agentic AI runtime** (92 source files, **736 tests, 0 failures**) implementing the full tool-calling loop—User → Assistant → [ToolUse → Hook → Permission → Execute → ToolResult]* → Response—using **sealed interfaces and pattern matching** to enforce exhaustive dispatch at compile time.
- Built a **hand-written SSE streaming parser** for chunked LLM API delivery and a **zero-dependency JSON parser/writer** for

session serialization, eliminating all runtime library dependencies from the hot path.

- Implemented **4 adaptive context compaction strategies** (SlidingWindow, PriorityRetention, EntropyPruning, LlmSummarization) with an entropy-based selector that automatically routes to the optimal strategy, sustaining long-running agentic sessions within fixed token budgets.
- Architected a **5-tier permission model** (ReadOnly → WorkspaceWrite → DangerFullAccess → Prompt → Allow), a **PreToolUse/PostToolUse hook pipeline** (subprocess + JSON-stdin + exit-code protocol), and an **MCP JSON-RPC 2.0 client** with stdio/HTTP transport for multi-provider LLM abstraction (Anthropic + OpenAI).

StreamForge: Zero-Dependency Video Streaming Platform [github.com/ndqkhanh/stream-forge] *Jan 2025*

Java 21, Virtual Threads, Adaptive Bitrate Streaming, Consistent Hashing, BM25, HTTP Range Requests | 6 modules, 127 tests

- Built a **6-module video streaming platform from scratch** in Java 21 with zero external dependencies, implementing **adaptive bitrate streaming** via buffer-based ABR with **EWMA bandwidth estimation and hysteresis** to prevent oscillation under fluctuating network conditions.
- Engineered a **CDN layer with consistent hashing** (150 virtual nodes, SHA-256) for zero-downtime node rebalancing and an **O(1) LRU cache**, plus chunk-based upload with **SHA-256 content-addressable deduplication** eliminating redundant storage across re-uploaded segments.
- Implemented **BM25 full-text search**, **HTTP range requests** for seek-accurate video delivery, a **priority-queue job scheduler** for transcoding pipeline fairness, and a **resolution ladder with pixel-ratio bitrate scaling** via Virtual Threads. **127 tests, 0 failures** across 6 modules.

TurboMQ: Elastic Distributed Message Queue [github.com/ndqkhanh/turbo-mq] *Nov 2024*

Java 21, gRPC, Raft Consensus, RocksDB, SWIM Gossip, Micrometer, Virtual Threads

- Designed a **distributed message queue** with a novel **per-partition Raft** model: each partition runs its own Raft group (inspired by CockroachDB's per-range Raft), enabling independent leader election and **zero-downtime partition migration**—unlike Kafka where controller failover blocks the entire cluster.
- **SWIM-lite gossip protocol** for cluster membership with incarnation-based conflict resolution, indirect probing for failure detection, and push-pull state dissemination. **Consumer group coordinator** with pluggable assignment strategies (Range, RoundRobin) and heartbeat-based session management.
- **Virtual-thread-per-partition** architecture: each of 10K+ partitions gets a dedicated lightweight thread. RocksDB storage with checkpoint-based snapshots sustaining **50K+ msg/sec** writes/node. **Raft snapshot transfer** for follower catch-up. Idempotent producer with sequence-number deduplication for **exactly-once delivery**.
- Prometheus metrics via Micrometer, gRPC API, Grafana cluster dashboard. **219 tests, 0 failures** across 6 modules.

FlashCache: Distributed In-Memory Cache [github.com/ndqkhanh/flash-cache] *Dec 2024*

Java 21, NIO Selector, RESP Protocol, HTTP/2, HPACK, TLS 1.3, SWIM Gossip, Consistent Hashing

- Built a **Redis-compatible distributed cache** from scratch with **6 protocol implementations**: RESP (Redis wire format), HTTP/1.1, **HTTP/2 binary framing with HPACK** header compression (RFC 7541), WebSocket (RFC 6455), and **TLS 1.3** termination via SSLEngine—all hand-built on Java NIO without Netty.
- **Consistent hash ring** with virtual nodes (SHA-256, `ConcurrentSkipListMap`) for zero-downtime cluster rebalancing. **SWIM gossip protocol** with incarnation-based conflict resolution, indirect probing, and push-pull state dissemination for decentralized membership.
- **LRU eviction** in O(1) via doubly-linked list + `ConcurrentHashMap`. TTL-based expiry. Primary-replica **replication** with sequence-based acks. Pub/Sub engine with glob-pattern subscriptions and WebSocket broadcast. **256 tests, 0 failures** across 6 modules.

GrabFlow: AI-Powered Distributed Ride-Sharing Platform [github.com/ndqkhanh/grabflow] *Mar 2025*

Java 21, Virtual Threads, H3 Geospatial, XGBoost, DSL Compiler, Saga Pattern | 7 microservices, 404 tests

- Built a **distributed ride-sharing platform** with 7 microservices (API Gateway, Location, Ride, Pricing, Payment, Notification) running on **4 custom infrastructure systems**: NexusDB (storage), TurboMQ (events), FlashCache (cache), AgentForge (agentic AI runtime)—all built from scratch.
- API Gateway with **from-scratch protocol stack**: IPv4/IPv6 dual-stack, **DNS resolver** (RFC 1035 wire protocol, pointer compression), TLS 1.3 with ALPN + cert hot-rotation, **Bloom filter** DDoS rate limiter (MurmurHash3, `AtomicLongArray`). Location Service with **H3 hexagonal geospatial index**, **CRC-32** packet integrity, **memory-mapped IPC** ring buffer, **hierarchical timing wheel** (O(1) scheduling). **404 tests, 0 failures** across 7 modules.

EDUCATION & AWARDS

University of Science — B.S. Computer Science (High-Quality Program) Oct 2020 – Oct 2024

GPA: 3.92/4.0 | **Salutatorian** (Top 2 in CS) | Scholarship every semester | Dean's List (2021–2023) | Bronze Medal, Le Hong Phong 30/4 Math Competition